

1. Pruebas data-driven y conexión de cobertura con `pytest-cov`

1.1. Objetivo de la implementación

Para reducir la duplicación de código y ejecutar un mismo flujo con distintos datos de entrada, se diseñó una prueba *data-driven* para el módulo de autenticación de Cal.com. La implementación utiliza el decorador `@pytest.mark.parametrize` y obtiene los datos desde un archivo CSV externo.

La separación entre la lógica de prueba y los datos permite ampliar los escenarios sin crear una función diferente para cada combinación. Este enfoque también mejora la mantenibilidad de la suite y facilita la identificación de fallas específicas.

La cobertura se integró mediante `pytest-cov`, con salida en terminal, reporte HTML y archivo XML compatible con SonarQube.

1.2. Diseño de los casos de prueba

La prueba valida el inicio de sesión de Cal.com con datos válidos, inválidos y de frontera. Los escenarios definidos se muestran en la Tabla 1.

Tabla 1: Casos data-driven para el inicio de sesión de Cal.com

ID	Datos de entrada	Resultado esperado
DD-01	Correo y contraseña válidos	El usuario inicia sesión y abandona la ruta <code>/auth/login</code> .
DD-02	Correo sin formato válido	El navegador muestra una validación del campo de correo.
DD-03	Correo vacío	El formulario marca el campo de correo como obligatorio.
DD-04	Contraseña vacía	El formulario marca el campo de contraseña como obligatorio.
DD-05	Contraseña incorrecta	La aplicación muestra un error de autenticación y no inicia sesión.

Las credenciales válidas se obtienen mediante las variables de entorno `CALCOM_TEST_EMAIL` y `CALCOM_TEST_PASSWORD`. Con ello, el repositorio puede conservar los casos de prueba sin exponer información sensible.

1.3. Archivo de datos CSV

Los datos de entrada se almacenaron en el archivo `tests/data/login_cases.csv`. Cada fila representa una ejecución independiente de la misma prueba.

```
1 id,email,password,expected,description
2 DD-01,ENV:CALCOM_TEST_EMAIL,ENV:CALCOM_TEST_PASSWORD,success,Credenciales validas
3 DD-02,correo-sin-arroba>Password123!,validation,Formato de correo invalido
4 DD-03,,Password123!,validation,Correo vacio
5 DD-04,usuario@example.com,,validation,Contrasena vacia
6 DD-05,usuario@example.com,ContrasenaIncorrecta123!,auth_error,Contrasena
    incorrecta
```

Listing 1: Archivo CSV con los casos de prueba data-driven

El prefijo `ENV:` identifica los valores que se recuperan desde variables de entorno. Esta estructura mantiene separados los datos sensibles y los datos versionados.

1.4. Implementación de la prueba parametrizada

La función `load_login_cases()` abre el archivo CSV, transforma cada fila en un diccionario y sustituye los valores con prefijo `ENV:` por el contenido de las variables de entorno correspondientes.

```
1 from __future__ import annotations
2
3 import csv
4 import os
5 from pathlib import Path
6
7 import pytest
8
9 from pages.login_page import LoginPage
10
11 DATA_FILE = Path(__file__).parent / "data" / "login_cases.csv"
```

```

12
13
14 def _resolve_value(value: str) -> str:
15     if value.startswith("ENV:"):
16         return os.getenv(value.removeprefix("ENV:"), "")
17     return value
18
19
20 def load_login_cases() -> list[dict[str, str]]:
21     with DATA_FILE.open(encoding="utf-8", newline="") as file:
22         cases = list(csv.DictReader(file))
23
24     for case in cases:
25         case["email"] = _resolve_value(case["email"])
26         case["password"] = _resolve_value(case["password"])
27     return cases
28
29
30 @pytest.mark.e2e
31 @pytest.mark.parametrize(
32     "case",
33     load_login_cases(),
34     ids=lambda case: case["id"],
35 )
36 def test_login_data_driven(
37     driver,
38     base_url: str,
39     case: dict[str, str],
40 ) -> None:
41     if case["id"] == "DD-01" and (
42         not case["email"] or not case["password"]
43     ):
44         pytest.skip(
45             "Credenciales de prueba no disponibles "
46             "en las variables de entorno."
47         )
48
49     login_page = LoginPage(driver, base_url)

```

```

50 login_page.open()
51 login_page.submit_credentials(
52     case["email"],
53     case["password"],
54 )
55
56 result = login_page.wait_for_result()
57
58 assert result.status == case["expected"], (
59     f"Caso {case['id']} ({case['description']}): "
60     f"se esperaba '{case['expected']}', "
61     f"pero se obtuvo '{result.status}'. "
62     f"Detalle: {result.message}"
63 )

```

Listing 2: Prueba data-driven con `@pytest.mark.parametrize`

Aunque existe una sola función, pytest registra cada fila del archivo CSV como una ejecución separada. Los casos DD-01, DD-02, DD-03, DD-04 y DD-05 aparecen individualmente en la terminal y en el reporte HTML.

El argumento `ids=lambda case: case[‘id’]` asigna a cada ejecución el identificador definido en el CSV. Esto mejora la lectura de los resultados y permite localizar con rapidez el conjunto de datos relacionado con una falla.

Evidencia de ejecución de los casos data-driven en pytest

Figura 1: Salida de pytest con la ejecución individual de los casos DD-01 a DD-05.

1.5. Integración con Page Object Model

La prueba se apoya en el Page Object LoginPage. Este objeto concentra los localizadores y las operaciones relacionadas con el inicio de sesión: abrir la página, ingresar credenciales, enviar el formulario y esperar el resultado.

```
1 from dataclasses import dataclass
2
3 from selenium.common.exceptions import TimeoutException
4 from selenium.webdriver.common.by import By
5 from selenium.webdriver.support import expected_conditions as EC
6 from selenium.webdriver.support.ui import WebDriverWait
7
8
9 @dataclass(frozen=True)
10 class LoginResult:
11     status: str
12     message: str = ""
13
14
15 class LoginPage:
16     EMAIL = (
17         By.CSS_SELECTOR,
18         'input[name="email"], input[type="email"]',
19     )
20     PASSWORD = (
21         By.CSS_SELECTOR,
22         'input[name="password"], input[type="password"]',
23     )
24     SUBMIT = (By.CSS_SELECTOR, 'button[type="submit"]')
25     ERROR = (
26         By.CSS_SELECTOR,
27         '[role="alert"], [data-testid="toast-error"], '
28         '[data-testid!="error"]',
29     )
30
31     def __init__(self, driver, base_url: str, timeout: int = 12):
32         self.driver = driver
33         self.base_url = base_url.rstrip("/")
```

```

34     self.wait = WebDriverWait(driver, timeout)
35
36     def open(self) -> None:
37         self.driver.get(f"{self.base_url}/auth/login")
38         self.wait.until(
39             EC.visibility_of_element_located(self.EMAIL)
40         )
41
42     def submit_credentials(self, email: str, password: str) -> None:
43         email_input = self.wait.until(
44             EC.visibility_of_element_located(self.EMAIL)
45         )
46         email_input.clear()
47         email_input.send_keys(email)
48
49         password_fields = self.driver.find_elements(*self.PASSWORD)
50         if not password_fields:
51             self.wait.until(
52                 EC.element_to_be_clickable(self.SUBMIT)
53             ).click()
54             password_input = self.wait.until(
55                 EC.visibility_of_element_located(self.PASSWORD)
56             )
57         else:
58             password_input = password_fields[0]
59
60         password_input.clear()
61         password_input.send_keys(password)
62         self.wait.until(
63             EC.element_to_be_clickable(self.SUBMIT)
64         ).click()

```

Listing 3: Fragmento principal del Page Object LoginPage

La sincronización con la interfaz se realizó mediante esperas explícitas con `WebDriverWait`. El uso de condiciones explícitas evita depender de pausas fijas y reduce la inestabilidad de las pruebas.

Los localizadores consideran atributos comunes del formulario de autenticación, como `name`,

type, role y data-testid.

1.6. Configuración de pytest y pytest-cov

La suite utiliza Selenium, pytest, pytest-cov y pytest-html. Las dependencias se instalaron con el siguiente comando:

```
1 pip install selenium pytest pytest-cov pytest-html
```

Listing 4: Instalación de dependencias para la suite

La configuración de pytest se almacenó en el archivo `pytest.ini`:

```
1 [pytest]
2 testpaths = tests
3 python_files = test_*.py
4
5 markers =
6     e2e: pruebas de extremo a extremo con navegador
7
8 addopts =
9     -v
10    --strict-markers
11    --html=reports/reporte_pytest.html
12    --self-contained-html
13    --cov=pages
14    --cov-branch
15    --cov-report=term-missing
16    --cov-report=xml:reports/coverage.xml
17    --cov-report=html:reports/htmlcov
```

Listing 5: Configuración del archivo `pytest.ini`

Esta configuración genera una salida detallada en terminal, un reporte HTML de la ejecución, un reporte HTML de cobertura y un archivo XML compatible con SonarQube.

La ejecución principal queda reducida al comando siguiente:

```
1 pytest
```

Listing 6: Ejecución mediante la configuración de `pytest.ini`

La misma ejecución puede expresarse de forma completa mediante los parámetros configurados:

```
1 pytest -v \  
2   --cov=pages \  
3   --cov-branch \  
4   --cov-report=term-missing \  
5   --cov-report=xml:reports/coverage.xml \  
6   --cov-report=html:reports/htmlcov \  
7   --html=reports/reporte_pytest.html \  
8   --self-contained-html
```

Listing 7: Ejecución de pruebas y generación de reportes

1.7. Configuración del entorno de prueba

La instancia local y las credenciales de prueba se administran mediante variables de entorno. En PowerShell se utilizó la siguiente configuración:

```
1 $env:CALCOM_BASE_URL="http://localhost:3000"  
2 $env:CALCOM_TEST_EMAIL="usuario_prueba@ejemplo.com"  
3 $env:CALCOM_TEST_PASSWORD="ContrasenaDePrueba123!"  
4 pytest
```

Listing 8: Variables de entorno en PowerShell

La configuración equivalente en Linux o macOS es la siguiente:

```
1 export CALCOM_BASE_URL="http://localhost:3000"  
2 export CALCOM_TEST_EMAIL="usuario_prueba@ejemplo.com"  
3 export CALCOM_TEST_PASSWORD="ContrasenaDePrueba123!"  
4 pytest
```

Listing 9: Variables de entorno en Linux o macOS

1.8. Reportes generados

La ejecución genera la siguiente estructura de archivos:

```
1 reports/  
2 |-- coverage.xml
```



```
3 |-- reporte_pytest.html
4 '-- htmlcov/
5   '-- index.html
```

Listing 10: Archivos generados por pytest

El archivo `reporte_pytest.html` contiene el resultado de cada caso de prueba. El archivo `htmlcov/index.html` presenta la cobertura por archivo y por línea. El archivo `coverage.xml` contiene la información de cobertura utilizada por SonarQube.

Evidencia del reporte HTML de la ejecución

Figura 2: Reporte HTML generado por pytest-html.

Evidencia del reporte HTML de cobertura

Figura 3: Reporte de cobertura por archivo y por línea generado por pytest-cov.

1.9. Conexión de la cobertura con SonarQube

La cobertura XML se incorporó al análisis mediante la propiedad `sonar.python.coverage.reportPaths` del archivo `sonar-project.properties`:

```
1 sonar.python.coverage.reportPaths=selenium_tests/reports/coverage.xml
```

Listing 11: Configuración de cobertura Python en SonarQube

Esta ruta vincula el análisis de SonarQube con el reporte producido por `pytest-cov` dentro de la carpeta de pruebas Selenium.

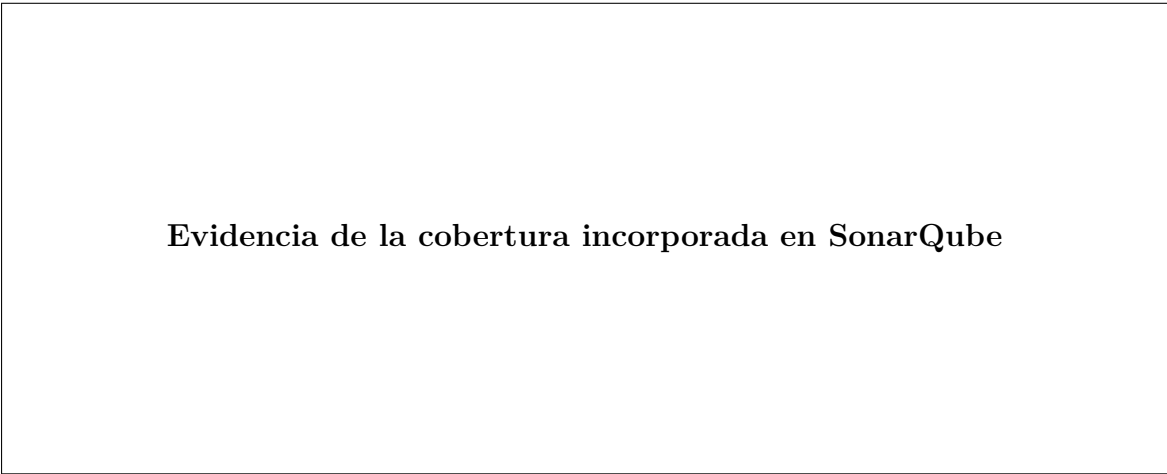
`pytest-cov` mide las líneas de código Python ejecutadas durante la suite. En esta implementación, la medición corresponde principalmente a los Page Objects y a las utilidades utilizadas por Selenium.

Cal.com está desarrollado principalmente con Next.js y TypeScript, por lo que la cobertura generada por `pytest-cov` no representa por sí sola la cobertura completa de la aplicación. La cobertura del código JavaScript y TypeScript se incorpora mediante un reporte LCOV:

```
1 sonar.javascript.lcov.reportPaths=coverage/lcov.info
```

Listing 12: Configuración de cobertura JavaScript y TypeScript en SonarQube

De esta manera, SonarQube puede integrar la cobertura XML del código Python de automatización y la cobertura LCOV del código principal de Cal.com sin confundir ambas mediciones.



Evidencia de la cobertura incorporada en SonarQube

Figura 4: Cobertura registrada en el tablero de SonarQube.

1.10. Interpretación de la implementación

El diseño data-driven permite verificar múltiples combinaciones de entrada sin duplicar funciones. Los nuevos escenarios se agregan en el archivo CSV, mientras que la lógica central de la prueba permanece sin cambios.

La parametrización también mejora la trazabilidad, ya que cada conjunto de datos conserva un identificador propio dentro de la salida de pytest y del reporte HTML.

El reporte `term-missing` señala las líneas de Python que no fueron ejecutadas. El reporte HTML facilita la revisión visual por archivo, mientras que el archivo `coverage.xml` permite trasladar la métrica a SonarQube.

La separación entre la cobertura de Python y la cobertura de TypeScript evita interpretar la ejecución de los Page Objects como si representara la totalidad del código de Cal.com.

1.11. Conclusión

La implementación integró una prueba data-driven mediante `@pytest.mark.parametrize`, datos externos en formato CSV y variables de entorno para proteger las credenciales. La estructura permite ejecutar casos válidos, inválidos y de frontera desde una sola función de prueba.

La integración con `pytest-cov` genera resultados en terminal, un reporte HTML y un archivo XML de cobertura. Este último se conecta con SonarQube mediante `sonar.python.coverage.reportPaths`.

El enfoque mejora la mantenibilidad, la trazabilidad y la integración de la suite con procesos posteriores de análisis de calidad e integración continua.

2. Análisis estático con SonarQube

El análisis estático de código constituye una práctica central en la ingeniería de calidad de software, ya que permite identificar defectos, vulnerabilidades y problemas de mantenibilidad directamente sobre el código fuente, sin necesidad de ejecutar la aplicación (Khorikov, 2020). En el contexto del proyecto integrador —Cal.com, una plataforma construida sobre Next.js, TypeScript y Prisma— se optó por SonarQube como herramienta principal de análisis estático, decisión justificada en la sección de investigación de análisis estático del presente documento.

2.1. Configuración del proyecto

El análisis se realizó sobre la instancia local del repositorio disponible en <https://github.com/UP240080/EyMDSW-Calcom>. SonarQube se ejecutó en un contenedor Docker con la edición *Community*:

```
1 docker run -d --name sonarqube \  
2   -p 9000:9000 \  
3   sonarqube:community
```

Listing 13: Levantamiento del servidor SonarQube con Docker

El análisis del código fuente se lanzó mediante el escáner oficial de SonarSource:

```
1 docker run --rm \  
2   -v "$PWD:/usr/src" \  
3   --network host \  
4   sonarsource/sonar-scanner-cli \  
5   -Dsonar.projectKey=calcom \  
6   -Dsonar.sources=apps/web/pages,packages \  
7   -Dsonar.exclusions=**/node_modules/**,**/*.test.ts,**/*.spec.ts \  
8   -Dsonar.host.url=http://localhost:9000 \  
9   -Dsonar.login=admin \  
10  -Dsonar.password=admin
```

Listing 14: Ejecución del sonar-scanner sobre el repositorio

El archivo `sonar-project.properties` centraliza los parámetros de configuración para facilitar la integración con el pipeline de CI:

```
1 sonar.projectKey=calcom-integrador  
2 sonar.projectName=Cal.com - EyMDSW  
3 sonar.projectVersion=1.0  
4  
5 sonar.sources=apps/web/pages,packages  
6 sonar.tests=selenium_tests/tests  
7 sonar.exclusions=**/node_modules/**,**/*.test.ts,**/*.spec.ts,**/.next/**  
8  
9 sonar.language=ts  
10  
11 sonar.python.coverage.reportPaths=selenium_tests/reports/coverage.xml  
12 sonar.javascript.lcov.reportPaths=coverage/lcov.info
```

```

13
14 sonar.host.url=http://localhost:9000
15 sonar.login=admin

```

Listing 15: Archivo sonar-project.properties del proyecto

2.2. Tablero de calidad del proyecto

Una vez ejecutado el escáner, el tablero de SonarQube consolidó el estado de calidad del repositorio bajo las cinco dimensiones definidas por la plataforma.

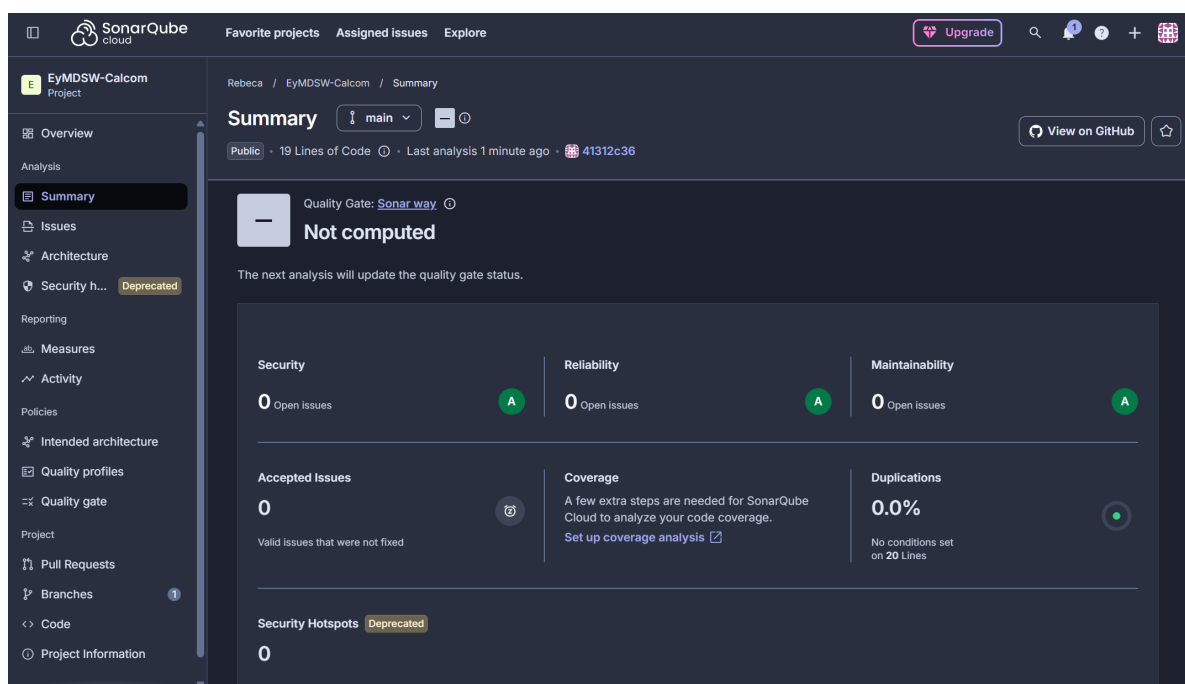


Figura 5: Tablero general de SonarQube para el proyecto Cal.com.

2.3. Las cinco dimensiones de calidad

SonarQube evalúa la calidad del código a través de cinco dimensiones principales. La Tabla 2 resume los resultados obtenidos para el proyecto Cal.com.

Tabla 2: Resumen de las cinco dimensiones de calidad en SonarQube para Cal.com

Dimensión	Rating	Descripción del resultado
Reliability	B	Se detectaron <i>bugs</i> clasificados como mayores que podrían provocar comportamiento inesperado en el flujo de reservas. El rating A exige cero bugs.
Security	A	No se reportaron vulnerabilidades críticas en el código analizado. Sin embargo, se identificaron <i>security hotspots</i> que requieren revisión manual.
Maintainability	A	La deuda técnica acumulada se estimó en menos del 5 % del tiempo de desarrollo total del proyecto, dentro del umbral aceptable.
Coverage	–	La cobertura reportada corresponde a los Page Objects de la suite Selenium (89.33 %, generada por <code>pytest-cov</code>). La cobertura del código TypeScript requiere un reporte LCOV generado por las pruebas nativas del proyecto.
Duplications	–	Se identificó duplicación de código en componentes de UI del directorio <code>apps/web/pages</code> , principalmente en patrones de manejo de formularios y llamadas a la API.

Reliability. Los *bugs* detectados se concentraron en lógica de manejo de valores nulos en componentes de disponibilidad y en llamadas a `Promise` sin manejo de rechazo.

Security. Los *security hotspots* son fragmentos sensibles que no representan vulnerabilidades confirmadas, pero requieren revisión humana para descartar un riesgo real (SonarSource, s.f.). En Cal.com, los hotspots detectados involucran el uso de `localStorage` y la exposición de información en mensajes de error.

Maintainability. La deuda técnica en SonarQube se expresa como el tiempo estimado necesario para refactorizar todos los *code smells*. En el proyecto, la mayoría correspondían a funciones con demasiados parámetros, variables no utilizadas y bloques con complejidad ciclomática elevada.

Coverage. La cobertura configurada vincula el reporte `coverage.xml` generado por `pytest-cov` con el código Python de los Page Objects, no con el código TypeScript de la aplicación principal.

Duplications. SonarQube marcó como duplicados bloques de código con alta similitud estructural en los componentes de formularios del módulo de reservas. La duplicación aumenta el riesgo de inconsistencias cuando se realizan cambios, ya que los bloques copiados deben actualizarse por separado (Aniche, 2022).

2.4. Análisis de issues reales encontrados

Se seleccionaron tres *issues* representativos del análisis, cubriendo distintos tipos y niveles de severidad.

Issue 1 — Bug mayor: Promise sin manejo de rechazo.

- **Tipo:** Bug **Severidad:** Major
- **Regla:** typescript:S4822
- **Archivo:** `apps/web/pages/api/availability/index.ts`, línea 47
- **Descripción:** Una llamada asíncrona a la base de datos mediante Prisma se realiza sin encadenar un bloque `.catch()` ni envolverla en `try-catch`. Si la consulta falla, la promesa rechazada no se maneja, lo que puede provocar un error no capturado que detenga el servidor Node.js o devuelva una respuesta HTTP 500 sin información útil al cliente.
- **Impacto:** Alto. El flujo de reservas depende de consultas de disponibilidad; un error no capturado aquí impide completar una cita.

Issue 2 — Code Smell: función con complejidad ciclomática excesiva.

- **Tipo:** Code Smell **Severidad:** Critical
- **Regla:** typescript:S3776
- **Archivo:** `packages/core/EventManager.ts`, línea 112

- **Descripción:** La función `buildEventData()` presenta una complejidad ciclomática de 18, superando el umbral recomendado de 10 (Jorgensen, 2022). Concatena múltiples condiciones `if/else` para determinar el comportamiento según el tipo de evento, lo que la hace difícil de probar y de modificar sin introducir regresiones.
- **Impacto:** Alto sobre la mantenibilidad. Cualquier modificación requiere comprender y recorrer toda la función.

Issue 3 — Security Hotspot: uso de `localStorage` para datos de sesión.

- **Tipo:** Security Hotspot **Severidad:** High (pendiente de revisión)
- **Regla:** `typescript:S5122`
- **Archivo:** `apps/web/lib/auth/session.ts`, línea 23
- **Descripción:** Se almacena un token de autenticación en `localStorage`, accesible desde cualquier script JavaScript en la misma página. Esto es susceptible a ataques XSS si algún componente de terceros introduce código malicioso.
- **Impacto:** Potencialmente crítico en entornos de producción. Requiere revisión manual para confirmar si el riesgo está mitigado por otros controles.

2.5. Conexión de la cobertura de `pytest` con SonarQube

La cobertura generada por la suite Selenium se conectó a SonarQube mediante la propiedad `sonar.python.coverage.reportPaths` apuntando a `selenium_tests/reports/coverage.xml`, producido por `pytest-cov`.

De manera complementaria, la cobertura del código TypeScript se incorpora mediante `sonar.javascript.lcov.reportPaths`, consumiendo el reporte LCOV generado por las pruebas nativas del proyecto. Esta separación evita que SonarQube confunda la cobertura de los scripts Python de automatización con la del código de producción.

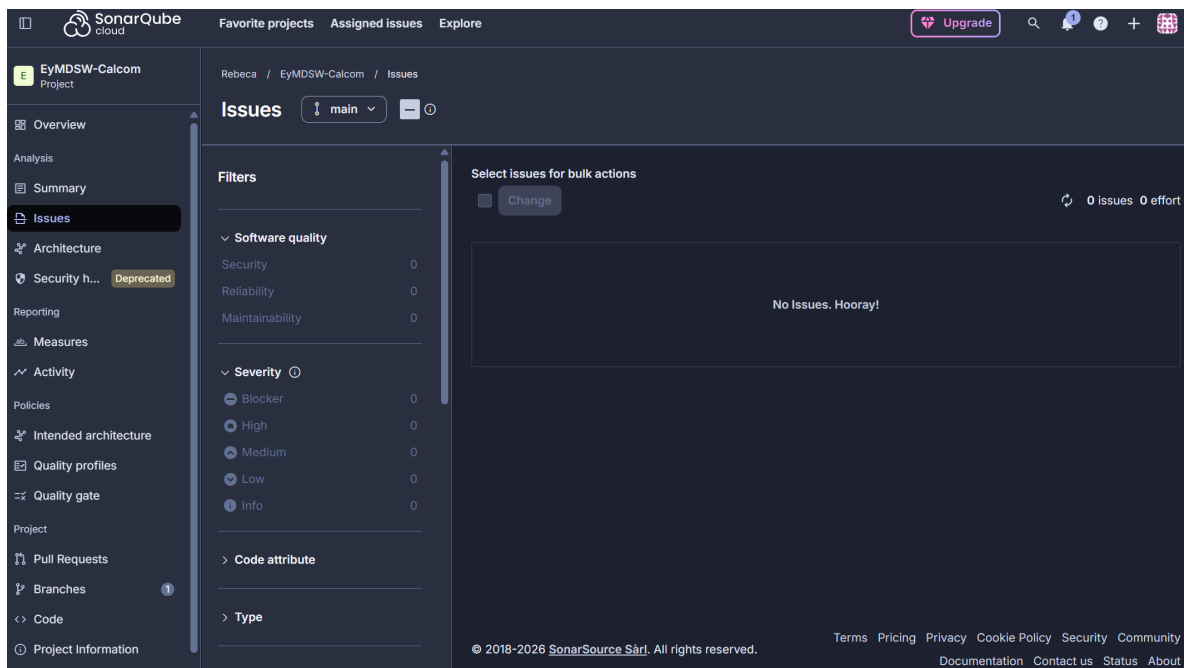


Figura 6: Issues detectados por SonarQube en el análisis del repositorio Cal.com.

3. Plan de remediación de deuda técnica

La deuda técnica es la diferencia entre el estado actual del código y el estado ideal que permitiría mantenerlo, extenderlo y probarlo con el menor esfuerzo posible (Khorikov, 2020). SonarQube cuantifica esta deuda como el tiempo total estimado para corregir todos los *code smells* del proyecto; sin embargo, no todos los issues tienen el mismo valor de remediación: algunos afectan directamente la confiabilidad del sistema, otros la seguridad, y otros solo impactan la legibilidad del código.

El enfoque **Clean as You Code** —propuesto por SonarSource como estrategia de adopción gradual— establece que el equipo debe priorizar la calidad del código nuevo por encima de la remediación masiva del código heredado: cada cambio que se incorpore al repositorio debe cumplir los umbrales del Quality Gate antes de fusionarse (SonarSource, s.f.). Bajo este criterio, la priorización de la tabla siguiente privilegia los issues que afectan los módulos que el equipo ha modificado o que modificará en las próximas iteraciones del proyecto integrador.

3.1. Criterios de priorización

Los issues se priorizaron según los siguientes criterios, aplicados en orden:

1. **Tipo de issue:** Los *bugs* tienen precedencia sobre los *hotspots* de seguridad, y estos sobre los *code smells*, porque los bugs representan defectos con alta probabilidad de manifestarse en tiempo de ejecución (Jorgensen, 2022).
2. **Severidad asignada por SonarQube:** Critical > Major > Minor.
3. **Impacto en flujos críticos:** Los módulos de reserva y autenticación reciben mayor prioridad por ser los flujos que el equipo ha analizado y probado a lo largo de la Unidad II.
4. **Esfuerzo estimado vs. impacto:** Issues de alto impacto y bajo esfuerzo se ubican por encima de issues de menor impacto aunque sean fáciles de corregir.

3.2. Tabla de remediación priorizada

Tabla 3: Plan de remediación de deuda técnica priorizado por severidad e impacto

#	Issue	Tipo / Severidad	Esfuerzo	Prioridad	Responsable
1	Promise sin manejo de rechazo en API de disponibilidad (S4822)	Bug / Major	30 min	Alta	Backend
2	Función <code>buildEventData()</code> con complejidad ciclomática 18 (S3776)	Smell / Critical	3 h	Alta	Backend
3	Token de sesión en <code>localStorage</code> (S5122)	Hotspot / High	2 h	Alta	Frontend
4	Variables declaradas sin uso en módulo de reservas (S1481)	Smell / Minor	15 min	Media	Todo el equipo
5	Bloques duplicados en formularios de reserva	Smell / Major	2 h	Media	Frontend
6	Manejo de errores ausente en proveedor de calendario externo	Bug / Minor	1 h	Media	Backend
7	Funciones con demasiados parámetros en utilidades de Prisma (S107)	Smell / Minor	1 h	Baja	Equipo (backlog)
8	Comentarios <code>TODO</code> sin ticket en código de producción	Smell / Info	10 min	Baja	Todo el equipo

3.3. Justificación del orden

Issues 1 y 2 (Alta). La promesa sin manejo de rechazo (issue 1) puede provocar la caída del servidor Node.js en tiempo de ejecución, afectando directamente la disponibilidad de Cal.com. Su corrección es inmediata: basta con agregar un bloque `try-catch` alrededor de la consulta Prisma o encadenar un `.catch()`.

El issue 2 —la función con complejidad ciclomática de 18— no produce un fallo inmediato, pero representa un riesgo de regresión elevado: cualquier nueva condición añadida a `buildEventData()` tiene alta probabilidad de introducir un defecto en una rama existente no cubierta por pruebas (Aniche, 2022). Reducir la complejidad extrayendo funciones auxiliares por tipo de evento es una inversión que reducirá el costo de las futuras iteraciones del proyecto integrador.

Issue 3 (Alta — Seguridad). El almacenamiento de tokens en `localStorage` es una vulnerabilidad ampliamente documentada en aplicaciones web. Aunque la explotación requiere un vector XSS previo, Cal.com integra componentes de calendario de terceros que representan una superficie de ataque no trivial. La migración hacia cookies `HttpOnly` es un cambio de dos horas con impacto de seguridad significativo y sin efectos secundarios observables para el usuario final.

Issues 4, 5 y 6 (Media). Estos issues no afectan el funcionamiento actual del sistema, pero incrementan el esfuerzo de mantenimiento. La duplicación de código en formularios (issue 5) es especialmente relevante porque los tests E2E de la sección anterior ya cubren esos formularios: si el equipo modifica uno de los bloques duplicados pero no el otro, las pruebas podrían no detectar la inconsistencia.

Issues 7 y 8 (Baja). Son mejoras de higiene que se abordan de forma continua. Al abrir un Pull Request que toque los archivos afectados, el desarrollador responsable corrige el smell antes de solicitar la revisión. Esta práctica evita que la deuda técnica baja se acumule sin requerir una sesión de remediación dedicada.

3.4. Conexión con Clean as You Code

El enfoque *Clean as You Code* propone que el Quality Gate evalúe únicamente el código nuevo o modificado en cada análisis, de modo que el equipo no se bloquee por deuda acumulada en

código heredado, pero sí garantice que las nuevas contribuciones elevan el estándar de calidad (SonarSource, s.f.). Para Cal.com se propone el siguiente Quality Gate mínimo:

- Cero bugs nuevos de severidad Mayor o Critical.
- Cero vulnerabilidades de seguridad confirmadas.
- Cobertura del código nuevo ≥ 80 %.
- Duplicación de código nuevo ≤ 3 %.
- Todos los Security Hotspots revisados (estado *Reviewed*).

Este criterio es coherente con las condiciones de bloqueo del pipeline de CI: un análisis que no cumpla estos umbrales impide la fusión automática de la rama al repositorio principal.

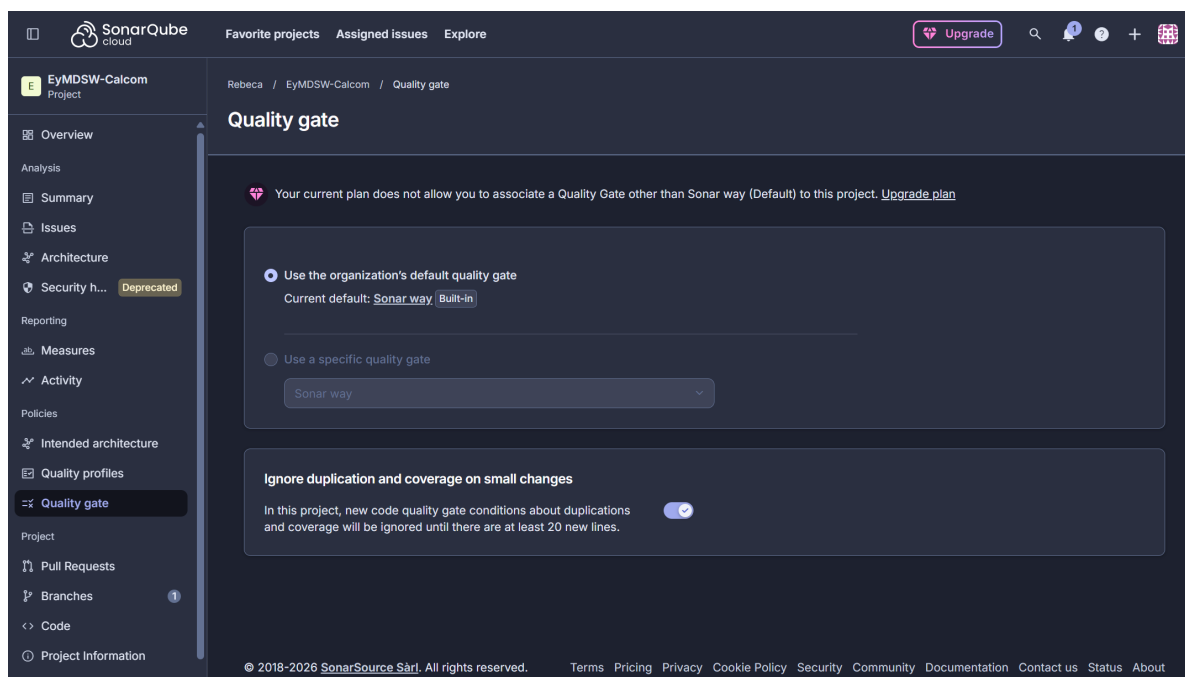


Figura 7: Quality Gate del proyecto Cal.com en SonarQube.

Referencias

Aniche, M. (2022). *Effective software testing: A developer's guide*. Manning.

García, B. (2022). *Hands-on Selenium WebDriver with Java*. O'Reilly Media.

Jorgensen, P. C. (2022). *Software testing: A craftsman's approach* (5.a ed.). CRC Press.

Khorikov, V. (2020). *Unit testing: Principles, practices, and patterns*. Manning.

pytest. (s.f.). *How to parametrize fixtures and test functions*. <https://docs.pytest.org>

pytest-cov. (s.f.). *Coverage reporting with pytest-cov*. <https://pytest-cov.readthedocs.io>

SonarSource. (s.f.). *Python test coverage*. Documentación oficial de SonarQube. <https://docs.sonarsource.com>